Ανατομία του Σθένους στο Spotify

Στην εργασία αυτή θα εξετάσετε τη μετρική του σθένους των μουσικών κομματιών στο Spotify.

Πάνος Λουρίδας, Αναπληρωτής Καθηγητής
Τμήμα Διοικητικής Επιστήμης και Τεχνολογίας
Οικονομικό Πανεπιστήμιο Αθηνών
louridas@aueb.gr

Το Spotify χρησιμοποιεί τη μετρική του *σθένους* (valence) που αντιστοιχεί στο πόσο χαρούμενο είναι ένα κομμάτι. Η μετρική αυτή δεν αναπτύχθηκε από το ίδιο το Spotify. Αναπτύχθηκε αρχικά από την Echo Nest, μια εταιρεία που αγοράστηκε από το Spotify το 2014. Δεν γνωρίζουμε ακριβώς πώς υπολογίζεται το σθένος κάθε κομματιού. Υπάρχουν μερικές λεπτομέρειες σε μία ανάρτηση σε ένα ιστολόγιο που μπορείτε να βρείτε εδώ:

https://web.archive.org/web/20170422195736/http://blog.echonest.com/post/66097438564/plotting-musics-emotional-valence-1950-2013 (https://web.archive.org/web/20170422195736/http://blog.echonest.com/post/66097438564/plotting-musics-emotional-valence-1950-2013)

Ο σκοπός σας είναι να μπορέσετε να βρείτε έναν τρόπο να υπολογίζετε το σθένος ενός κομματιού. Για το σκοπό αυτό θα πρέπει να χρησιμοποιήσετε την προγραμματιστική διεπαφή του Spotify (Spotify API), ακολουθώντας τις οδηγίες εδώ: https://developer.spotify.com/ (https://developer.spotify.com/). Μπορείτε να χρησιμοποιήσετε τις παρακάτω δύο κλήσεις της διεπαφής για τη συγκέντρωση δεδομένων που θα επεξεργαστείτε:

- Get Track's Audio Features (https://developer.spotify.com/documentation/web-api/reference/#/operations/get-audio-features) και Get Tracks' Audio Features (https://developer.spotify.com/documentation/web-api/reference/#/operations/get-several-audio-features).
- Προαιρετικά, Get Track's Audio Analysis (https://developer.spotify.com/documentation/web-api/reference/#/operations/get-audio-analysis).

Ως βάση θα χρησιμοποιήσετε τα δεδομένα για κομμάτια που έχουν βρεθεί στις κορυφαίες θέσεις ακουσμάτων σε διάφορες χώρες, και τα οποία μπορείτε να βρείτε εδώ: https://doi.org/10.5281/zenodo.4778562 (https://doi.org/10.5281/zenodo.4778562) και στο αρχείο `charts.zip` .

Περιεχόμενα

- Λήψη δεδομένων
- Ερώτηση 1
  - Επεξεργασία δεδομένων
  - 1η Μέθοδος: Αλγόριθμος Πλησιέστερων Γειτόνων (k-NN)
  - 2η Μέθοδος: Δένδρο Αποφάσεως
  - 3η Μέθοδος: Αφελής Ταξινομητής Bayes (Gaussian Naive Bayes)
  - Συμπέρασμα
- Ερώτηση 2

Τελικά συμπεράσματα

Λήψη δεδομένων

Αποσυμπιέζω το *charts.zip* μέσα στον φάκελο *assignment3*, οπότε τα δεδομένα βρίσκονται οργανωμένα στον φάκελο *assignment3/Charts*.

Για την συλλογή όλων των απαραίτητων δεδομένων εκπαίδευσης και εξέτασης, ακολουθώ την εξής διαδικασία:

- Συγκέντρωση όλων των διακριτών ids κομματιών από τα αρχεία. Το αρχείο *spotify_ids.txt*, το οποίο χρησιμοποιείται ως test set, περιέχει περίπου 1200 κομμάτια, όπως δείχνω παρακάτω. Για αυτό θέλω το train set να έχει περίπου τετραπλάσιο μέγεθος, ώστε η αναλογία συνολικά να είναι 80-20. Επιλέγω να χρησιμοποιήσω τα δεδομένα από τα charts Η.Π.Α. και Μ. Βρετανίας, τα οποία συνδυασμένα έχουν περίπου το επιθυμητό μέγεθος.
- Λήψη των πληροφοριών για κάθε ένα εξ αυτών από την API του Spotify. Έκανα εγγραφή στο Spotify και καταχώρησα την παρούσα «εφαρμογή», ώστε να λάβω client id και client secret. Επίσης, στις ρυθμίσεις της καταχωρημένης στο dashboard εφαρμογής, συμπλήρωσα το Redirect URI με την διεύθυνση στην οποία τρέχει η εφαρμογή (εν προκειμένω το jupyter, σε κάποια θύρα του τοπικού δικτύου).Τα υπόλοιπα βήματα προς τον σκοπό αυτόν εξηγούνται αναλυτικά στην συνέχεια.

Περιεχόμενα φακέλλου *Charts*

In [8]:
```
1 os.listdir()
```

Out[8]:
```
['2017', '2018', '2019']
```

Συγκομιδή διακριτών ids με χρήση του `set`

In [36]:
```
import os
import pandas as pd

##os.chdir("./Charts/")

song_ids = set()
for country in ["us", "gb"]:
    for year in os.listdir(f"./{country}"):
        for file in os.listdir(f"./{country}/{year}"):
            week = pd.read_csv(f"./{country}/{year}/{file}", sep = '\t', usecols = ["song_id"])
            week = week["song_id"].tolist()
            song_ids.update(week)
```

Ενδεικτικά η μορφή ενός αρχείου (γιά μία εβδομάδα). Τα αρχεία csv χρησιμοποιούν τον χαρακτήρα tab για διαχωρισμό των στηλών.

In [22]:
```
week.head()
```

Out[22]:

	position	song_id	song_name	artist	streams	last_week_position	weeks_on_chart	peak_position	position_status
0	1	2rPE9A1vEgShuZxxzR2tZH	thank u, next	Ariana Grande	31187542	3.0	9	1	2
1	2	3KkXRkHbMCARz0aVfEt68P	Sunflower - Spider-Man: Into the Spider-Verse	Post Malone	30229896	5.0	3	2	3
2	3	6MWtB6iiXyIwun0YzU6DFP	Wow.	Post Malone	29083499	25.0	2	3	22
3	4	4w8niZpiMy6qz1mntFA5uM	Taki Taki (with Selena Gomez, Ozuna & Cardi B)	DJ Snake	24784744	6.0	14	1	2
4	5	5p7ujcrUXASCNwRaWNHR1C	Without Me	Halsey	22862806	11.0	13	4	6

In [37]:
```
len(song_ids)
```

Out[37]:
```
4961
```

Μέγεθος test set

In [30]:
```
with open("spotify_ids.txt", 'r') as f:
    for i, _ in enumerate(f):
        pass
    print(i + 1)
```

```
1162
```

Για την επικοινωνία με το API του Spotify χρειάζομαι επίσης την βιβλιοθήκη `requests` .

In [ ]:
```
##!pip install requests ## δεν έχω τρέξει το κελί γιατί την είχα ήδη εγκατεστημένη
```

In [66]:
```
CLIENT_ID = "d71e5ecc8abb43f58d0a21c026e73f0c"
CLIENT_SECRET = "ff5a2e92aaf14a7da28f08e5444dbafd"
```

Κατασκευή base-64 client credentials

In [67]:
```
import base64
```

In [68]:
```
client_creds = f"{CLIENT_ID}:{CLIENT_SECRET}"
client_creds_b64 = base64.b64encode(client_creds.encode())
print(client_creds_b64)
```

```
b'ZDcxZTVlY2M4YWJiNDNmNThkMGEyMWMwMjZlNzNmMGM6ZmY1YTJlOTJhYWYxNGE3ZGEyOGYwOGU1NDQ0ZGJhZmQ='
```

Λήψη access token με αποστολή POST request στο κατάλληλο endpoint. Κάθε τέτοιο token διαρκεί για 3600 δευτερόλεπτα - μία ώρα.

In [69]:
```
import requests
token_url = "https://accounts.spotify.com/api/token/"
token_data = {
    "grant_type": "client_credentials"
}
token_headers = {
    "Authorization": f"Basic {client_creds_b64.decode()}"
}
```

In [70]:
```
r = requests.post(token_url, data = token_data, headers = token_headers)
print(r.json())
```

```
{'access_token': 'BQA0zsU_onn_XZWPKiBgDvsPNjHRW2EeoOs22hNHqu19uYPy83VnplaxN4iirnMgMvaTyFC3Xs9qE1d9PlM', 'token_type': 'Bearer', 'expires_in': 3600}
```

In [71]:
```
access_token = r.json()["access_token"]
```

Παράδειγμα απόκρισης

In [6]:
```
url = "https://api.spotify.com/v1/audio-features/7qiZfU4dY1lWllzX7mPBI3"
headers = {
    "Authorization": "Bearer BQCHvIPY6sTDwB-BihN2ku1WYL7OZ64T7Hfj075HAjyQ8hVimJarN8nXdvWMkKQqorxNAUA9Fwn-a5fu20w",
    "Content-Type": "application/json"
}
r = requests.get(url, headers = headers)
print(r.json())
```

```
{'danceability': 0.825, 'energy': 0.652, 'key': 1, 'loudness': -3.183, 'mode': 0, 'speechiness': 0.0802, 'acousticness': 0.581, 'ins
trumentalness': 0, 'liveness': 0.0931, 'valence': 0.931, 'tempo': 95.977, 'type': 'audio_features', 'id': '7qiZfU4dY1lWllzX7mPBI3',
'uri': 'spotify:track:7qiZfU4dY1lWllzX7mPBI3', 'track_href': 'https://api.spotify.com/v1/tracks/7qiZfU4dY1lWllzX7mPBI3', 'analysis_u
rl': 'https://api.spotify.com/v1/audio-analysis/7qiZfU4dY1lWllzX7mPBI3', 'duration_ms': 233713, 'time_signature': 4}
```

In [25]:
```
all_responses = []
```

Λήψη δεδομένων κομματιών μουσικής από το δεύτερο endpoint που αναφέρεται στην εκφώνηση, που επιστρέφει μαζικά.

In [44]:
```
headers = {
    "Authorization": f"Bearer {access_token}",
    "Content-Type": "application/json"
}

url = "https://api.spotify.com/v1/audio-features"
song_ids_list = list(song_ids)
for i in range(0, len(song_ids_list) - 100, 100): ## όριο 100 ανά request
    params = {
        "ids": ','.join(song_ids_list[i: i + 100])
    }
    r = requests.get(url, headers = headers, params = params)
    if(r.status_code != 200):
        print(r.json())
        break
    for s in r.json()["audio_features"]: all_responses.append(s)
```

Και τα υπόλοιπα που δεν χώρεσαν στο τελευταίο παράθυρο μήκους 100

In [52]:
```
headers = {
    "Authorization": f"Bearer {access_token}",
    "Content-Type": "application/json"
}

song_ids_list = list(song_ids)
params = {
    "ids": ','.join(song_ids_list)
}
r = requests.get(url, headers = headers, params = params)
if(r.status_code != 200):
    print(r.json())
for s in r.json()["audio_features"]: all_responses.append(s)
```

In [53]:
```
len(all_responses)
```

Out[53]:
```
4961
```

Έχω πλέον τις πληροφορίες για όλα τα κομμάτια του αρχικού συνόλου

Παρακάτω αποθηκεύω τοπικά τα αρχεία σε σειριοποιημένη μορφή με την βιβλιοθήκη `pickle` ώστε να είναι προσβάσιμα μεταξύ πολλών εκτελέσεων του notebook και να μην χρειαστεί ξανά επικοινωνία με την API.

Αυτό δημιουργεί περισσότερα αρχεία, τα οποία όμως **δεν** είναι αναγκαία για την εκτέλεση της εργασίας αυτής.

In [ ]:
```
import pickle
```

In [49]:
```
with open("songids.pkl", "wb") as f:
    pickle.dump(song_ids, f)
```

In [24]:
```
with open("songids.pkl", "rb") as f:
    song_ids = pickle.load(f)
```

In [56]:
```
with open("all_responses.pkl", "wb") as f:
    pickle.dump(all_responses, f)
```

In [2]:
```
with open("all_responses.pkl", "rb") as f:
    all_responses = pickle.load(f)
```

Για τον ίδιο λόγο αποθηκεύω τα δεδομένα και σε αρχείο csv με την pandas .

In [3]:
```
import pandas as pd

columns = list(all_responses[0].keys())

df = pd.DataFrame(columns = columns)
```

In [4]:
```
for i in all_responses:
    df = df.append(i, ignore_index = True)
```

In [5]:
```
df.head()
```

Out[5]:

	danceability	energy	key	loudness	mode	speechiness	acousticness	instrumentalness	liveness	valence	tempo	type	id	
0	0.599	0.317	1	-11.212	1	0.1740	0.7480	0	0.1110	0.203	148.090	audio_features	0Xcy81PsJCOO6mhLZaQyQ4	sp
1	0.865	0.624	8	-8.224	1	0.1030	0.0330	0	0.0984	0.396	94.940	audio_features	2iWEpiPiIPf5D4f2UGBlrZ	
2	0.708	0.235	11	-16.156	0	0.3180	0.7720	0.00238	0.1030	0.169	139.998	audio_features	5SFTZsHW3HdCp9f1zbzitS	
3	0.575	0.571	1	-7.906	1	0.0439	0.0558	0	0.0912	0.449	105.049	audio_features	4NBTZtAt1F13VvlSKe6KTl	
4	0.890	0.624	1	-3.966	1	0.1030	0.0937	0.000005	0.0903	0.762	116.056	audio_features	5xesHO6GLWTrjsKOjAXRmf	

In [11]:
```
df.to_csv("train.csv")
```

Επαναλαμβάνω την ίδια διαδικασία για το test set, που προέρχεται από το αρχείο *spotify_ids.txt*. Υποθέτω ότι δεν περιέχει κομμάτια που υπήρχαν και στα δεδομένα εκπαίδευσης, αφού αυτά καθορίζονται από την εκφώνηση.

In [64]:
```
test_ids = []
test_responses = []
with open("spotify_ids.txt", 'r') as f:
    for line in f:
        test_ids.append(line.rstrip('\n')) ## slow but I don't have to deal with trailing \n
```

In [73]:
```
headers = {
    "Authorization": f"Bearer {access_token}",
    "Content-Type": "application/json"
}

url = "https://api.spotify.com/v1/audio-features"
for i in range(0, len(test_ids) - 100, 100): ## όριο 100 ανά request
    params = {
        "ids": ','.join(test_ids[i: i + 100])
    }
    r = requests.get(url, headers = headers, params = params)
    if(r.status_code != 200):
        print(r.json())
        break
    for s in r.json()["audio_features"]: test_responses.append(s)
```

In [81]:
```
params = {
    "ids": ','.join(test_ids[-62:])
}
r = requests.get(url, headers = headers, params = params)
if(r.status_code != 200):
    print(r.json())
for s in r.json()["audio_features"]: test_responses.append(s)
```

In [74]:
```
len(test_ids)
```

Out[74]: 1162

In [82]:
```
len(test_responses)
```

Out[82]: 1162

In [83]:
```
test_df = pd.DataFrame(columns = columns)
for i in test_responses:
    test_df = test_df.append(i, ignore_index = True)
```

In [84]:
```
test_df.head()
```

Out[84]:

	danceability	energy	key	loudness	mode	speechiness	acousticness	instrumentalness	liveness	valence	tempo	type	id
0	0.585	0.436	10	-8.761	1	0.0601	0.72100	0.000013	0.1050	0.132	143.874	audio_features	7IPN2DXiMsVn7XUKtOW1CS
1	0.680	0.826	0	-5.487	1	0.0309	0.02120	0.000012	0.5430	0.644	118.051	audio_features	5QO79kh1waicV47BqGRL3g
2	0.514	0.730	1	-5.934	1	0.0598	0.00146	0.000095	0.0897	0.334	171.005	audio_features	0VjIjW4GlUZAMYd2vXMi3b
3	0.731	0.573	4	-10.059	0	0.0544	0.40100	0.000052	0.1130	0.145	109.928	audio_features	4MzXwWMhyBbmu6hOcLVD49
4	0.907	0.393	4	-7.636	0	0.0539	0.45100	0.000001	0.1350	0.202	104.949	audio_features	5Kskr9LcNYa0tpt5f0ZEJx

In [85]:
```
test_df.to_csv("test.csv")
```

Ερωτήσεις

E1: Πρόβλεψη Σθένους άνευ Νευρωνικών Δικτύων

Χρησιμοποιήστε τεχνικές Μηχανικής Μάθησης, της επιλογής σας, αλλά εκτός νευρωνικών δικτύων, για να προβλέψετε με τον καλύτερο τρόπο το σθένος των κομματιών.

- Θα χρησιμοποιήσετε τουλάχιστον τρεις διαφορετικές μεθόδους. Για κάθε μέθοδο αναζητείστε το καλύτερο μοντέλο με τις καλύτερες δυνατές τιμές υπερπαραμέτρων.
- Αφού βρείτε το καλύτερο μοντέλο, εξηγείστε, όσο είναι δυνατόν, ποια χαρακτηριστικά επηρεάζουν το σθένος ενός κομματιού.
- Αφού βρείτε το καλύτερο μοντέλο, θα το χρησιμοποιήσετε για να υπολογίσετε το σθένος των κομματιών που δίνονται στο αρχείο `spotify_ids.txt` . Ως μετρική θα πρέπει να χρησιμοποιήσετε το Μέσο Απόλυτο Λάθος (Mean Absolute Error).

Επεξεργασία Δεδομένων

- Αφαιρώ τα χαρακτηριστικά που δεν θεωρώ χρήσιμα: mode, type, id, uri, track_href, analysis_url, time_signature
- Προκειμένου να μπορέσω να εφαρμόσω αλγορίθμους κατηγοριοποίησης που παρουσιάσθηκαν στο μάθημα, όπου είναι παραίτητο ομαδοποιώ - κβαντίζω τις τιμές της στήλης valence σε διαστήματα πλάτους 0.1. Οι τιμές κυμαίνονται μεταξύ 0 και 1 όπως αναφέρεται στην τεκμηρίωση.
- Σε όποιες μεθόδους χρειάζεται, κανονικοιποιώ τις τιμές των χαρακτηριστικών.
- Πραγματοποιώ ανάλυση κυρίων συνιστωσών.

In [42]:
```
df = pd.read_csv("train.csv")
```

In [44]:
```
df.drop(columns = ["mode", "type", "id", "uri", "track_href", "analysis_url", "time_signature"], inplace = True)
```

In [11]:
```
import numpy as np

## η παρακάτω κβάντιση της στήλης valence εφαρμόζεται μόνον όπου το πρόβλημα προσεγγίζεται ως πρόβλημα ταξινόμησης
df["valence"] = pd.np.digitize(df["valence"], bins = [x / 10 for x in range(0, 11)])
```

In [12]:
```
df.head()
```

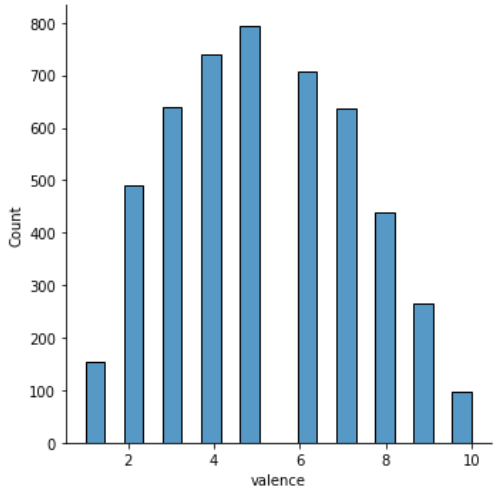
Out[12]:

	danceability	energy	key	loudness	speechiness	acousticness	instrumentalness	liveness	valence	tempo	duration_ms
0	0.599	0.317	1	-11.212	0.1740	0.7480	0	0.1110	3	148.090	215413
1	0.865	0.624	8	-8.224	0.1030	0.0330	0	0.0984	4	94.940	97344
2	0.708	0.235	11	-16.156	0.3180	0.7720	0.00238	0.1030	2	139.998	120059
3	0.575	0.571	1	-7.906	0.0439	0.0558	0	0.0912	5	105.049	197947
4	0.890	0.624	1	-3.966	0.1030	0.0937	0.000005	0.0903	8	116.056	169814

In [30]:
```
import seaborn as sns

sns.displot(df["valence"])
```

Out[30]:
```
<seaborn.axisgrid.FacetGrid at 0x1fe1de633a0>
```



Με τον τρόπο που ορίσθηκαν οι δέκα κλάσεις, τα δείγματα δεν είναι κατανεμημένα ομοιόμορφα σε αυτές.

Principal Component Analysis

In [15]:
```
components = df.columns.values.tolist()
components.remove("valence")
print(components)
```

```
['danceability', 'energy', 'key', 'loudness', 'speechiness', 'acousticness', 'instrumentalness', 'liveness', 'tempo', 'duration_ms']
```

In [16]:
```
components_df = df[components]
```

In [17]:
```
from sklearn.preprocessing import StandardScaler

standard_scaler = StandardScaler()
components_standard_scaled = standard_scaler.fit_transform(components_df)
```

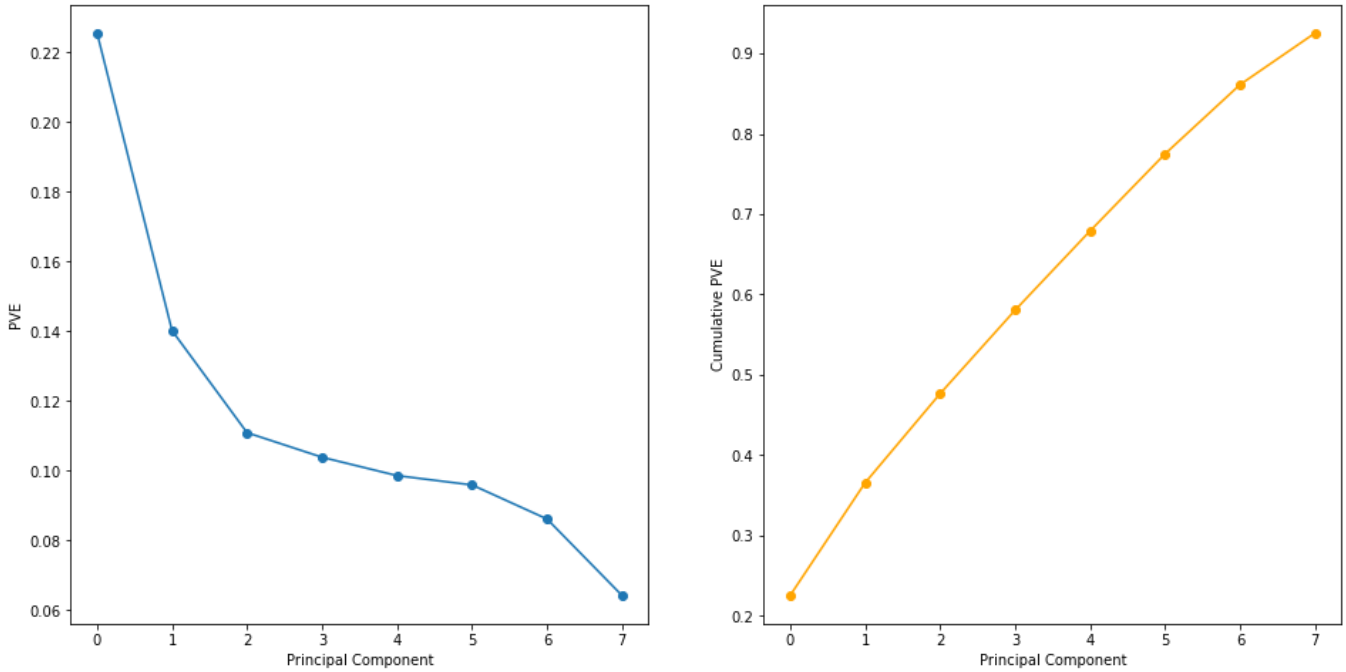
In [27]:
```
from sklearn.decomposition import PCA

pca = PCA(n_components = 8)
pca.fit(components_standard_scaled)
```

Out[27]:
```
PCA(n_components=8)
```

In [28]:
```
import matplotlib.pyplot as plt

plt.figure(figsize = (16, 8))
plt.subplot(1, 2, 1)
plt.plot(pca.explained_variance_ratio_, marker = 'o')
plt.xlabel('Principal Component')
plt.ylabel('PVE')
plt.subplot(1, 2, 2)
plt.plot(np.cumsum(pca.explained_variance_ratio_), marker = 'o', color = 'orange')
plt.xlabel('Principal Component')
_ = plt.ylabel('Cumulative PVE')
```



Ύστερα από δοκιμές διαφόρων τιμών συνιστωσών, παρατηρώ ότι η ελάχιστη που επιτρέπει την εξήγηση ικανοποιητικού ποσοστού της διακύμανσης είναι 8. Εφαρμόζοντας αυτήν την τεχνική, συνεπώς, θα μείωνα τις αρχικές διστάσεις κατά 20%. Ωστόσο, δεδομένου ότι τα αρχικά χαρακτηριστικά είναι μόλις 10, δεν αξίζει αυτή η μικρή απώλεια πληροφορίας, και τελικά **δεν** εφαρμόζω PCA για την εργασία αυτήν.

Πρώτη Μέθοδος: Αλγόριθμος k Nearest Neighbors

Σημείωση: αφού πήρα την απόφαση να μετατρέψω το πρόβλημα παλινδρόμησης σε πρόβλημα ταξινόμησης, προφανώς **δεν μπορώ να χρησιμοποιήσω την μετρική Mean Absolute Error για αυτό το μοντέλο**. Θα χρησιμοποιήσω την accuracy ως γνώμονα επιλογής βέλτιστων υπερπαραμέτρων. Για την αξιολόγηση κάθε μοντέλου, εκτός από αυτήν θα ληφθεί υπ' όψιν και ο χρόνος εκπαίδευσης.

In [40]:
```
labels = df["valence"].values
features = components_standard_scaled
```

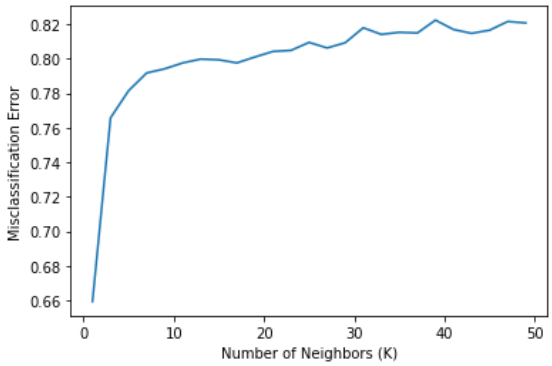
In [41]:
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

# Εξετάζω μόνο τα περιττά k από το 11 έως το 50
neighbors = list(range(1, 50, 2))
# empty list that will hold cv scores
cv_scores = []
# perform 10-fold cross validation
for k in neighbors:
    knn = KNeighborsClassifier(n_neighbors = k)
    scores = cross_val_score(knn, features, labels, cv = 10, scoring = 'accuracy')
    cv_scores.append(scores.mean())

# το σφάλμα είναι το αντίστροφο της πιστότητας
mean_error = [1 - x for x in cv_scores]

# plot misclassification error vs k
plt.plot(neighbors, mean_error)
plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Misclassification Error")
plt.show()

# determining best k
optimal_k = neighbors[mean_error.index(min(mean_error))]
print("The optimal number of neighbors (calculated in the training set) is %d" % optimal_k)
```




```
The optimal number of neighbors (calculated in the training set) is 1
```

Φαίνεται πως η μέθοδος αυτή δεν είναι καθόλου αποτελεσματική. Η ορθότητα που επιτυγχάνεται είναι μεν καλύτερη από την τυχαία μαντεψιά (10%), αλλά τόσο χαμηλή που δεν έχει νόημα ούτε η εξέταση του πίνακα συγχύσεως.

Δεύτερη Μέθοδος: Δένδρο Αποφάσεως

Τα δένδρα αποφάσεως επιλύουν και προβλήματα παλινδρόμησης, άρα εδώ χρησιμοποιώ τα αρχικά δεδομένα, και επιπλέον δεν κανονικοποιώ.

In [49]:
```
X_train = df.drop(columns = "valence").values
Y_train = df["valence"].values
```

In [55]:
```
from sklearn.tree import DecisionTreeRegressor

dtree = DecisionTreeRegressor(max_depth = 4)

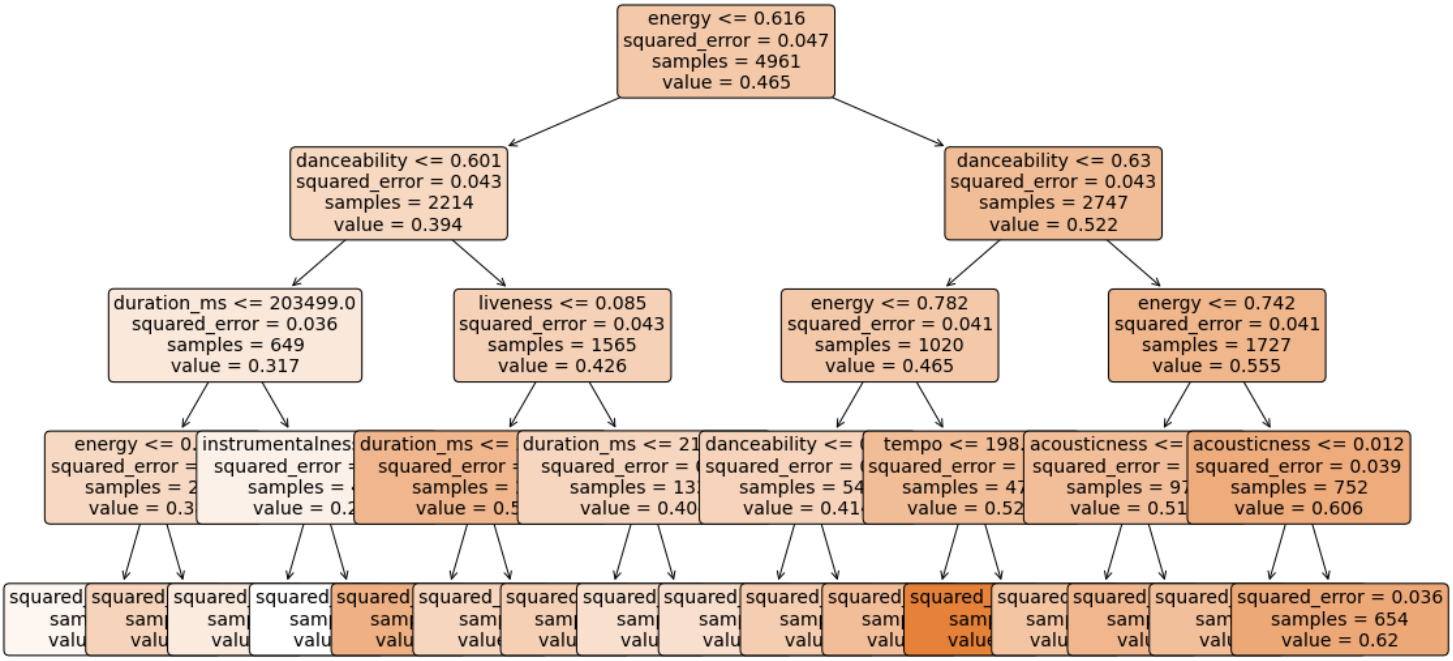
dtree.fit(X_train, Y_train)
```

Out[55]:
```
DecisionTreeRegressor(max_depth=4)
```

Η επιλογή τιμής βάθους ήταν αυθαίρετη. Η τιμή πρέπει να είναι αρκετά μεγάλη ώστε να γίνει ουσιώδης διαχωρισμός των διαστημάτων, αλλά όχι τόσο μεγάλη ώστε να γίνει υπερπροσαρμογή.

In [56]:
```
from sklearn import tree

plt.figure(figsize = (18, 10))
_ = tree.plot_tree(dtree,
                   feature_names = df.drop(columns = "valence").columns,
                   filled = True, rounded = True, fontsize = 14)
```



In [86]:
```
X_test = test_df[components].values
Y_test = test_df["valence"].values
```

In [87]:
```
from sklearn import metrics

predicted = dtree.predict(X_test)
print(metrics.mean_absolute_error(Y_test, predicted))
```

```
0.1708138604423224
```

Αυτή η σχετικά χαμηλή τιμή είναι σημαντική βελτίωση σε σχέση με το πρώτο πείραμα.

Τρίτη Μέθοδος: Αφελής Ταξινομητής Bayes

Πάλι το προσεγγίζω ως πρόβλημα ταξινόμησης, άρα χρησιμοποιώ τα αντίστοιχα μετασχηματισμένα δεδομένα.

In [92]:
```
features = df[components].values
labels = np.digitize(df["valence"], bins = [x / 10 for x in range(0, 11)])
```

In [93]:
```
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import make_pipeline

model = make_pipeline(StandardScaler(), PCA(n_components = 8), GaussianNB())
model.fit(features, labels)
```

Out[93]:
```
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('pca', PCA(n_components=8)), ('gaussiannb', GaussianNB())])
```

In [96]:
```
X_test = test_df[components].values
Y_test = np.digitize(test_df["valence"], bins = [x / 10 for x in range(0, 11)])
```

In [97]:
```
preds = model.predict(X_test)
```

In [100]:
```
from sklearn.metrics import classification_report

print(classification_report(Y_test, preds))
```

```
              precision    recall  f1-score   support

           1       0.14      0.18      0.16        28
           2       0.18      0.19      0.18       105
           3       0.20      0.09      0.13       117
           4       0.12      0.09      0.10       127
           5       0.16      0.55      0.25       186
           6       0.10      0.03      0.05       162
           7       0.16      0.14      0.15       159
           8       0.09      0.02      0.04       127
           9       0.00      0.00      0.00        98
          10       0.50      0.02      0.04        53

    accuracy                           0.16      1162
   macro avg       0.17      0.13      0.11      1162
weighted avg       0.15      0.16      0.12      1162
```

```
c:\users\kyriakos\appdata\local\programs\python\python39\lib\site-packages\sklearn\metrics\_classification.py:1308: UndefinedMetricW
arning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` paramete
r to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
c:\users\kyriakos\appdata\local\programs\python\python39\lib\site-packages\sklearn\metrics\_classification.py:1308: UndefinedMetricW
arning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` paramete
r to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
c:\users\kyriakos\appdata\local\programs\python\python39\lib\site-packages\sklearn\metrics\_classification.py:1308: UndefinedMetricW
arning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` paramete
r to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```

Ξανά, δεν έχει νόημα να εξετάσω πίνακα συγχύσεως, για τον ίδιο λόγο.

Συμπέρασμα

Το δένδρο αποφάσεων είχε με διαφορά την καλύτερη επίδοση, μάλλον επειδή ήταν η μόνη μέθοδος στην οποία δεν κβαντίστηκαν οι τιμές του σθένους. Δεν συγκρίνεται άμεσα η επίδοση βέβαια, αφού στο μεν δένδρο μετρώ MAE, και στις άλλες δύο classification accuracy, αλλά η διαφορά στην χρησιμότητα των μοντέλων είναι εμφανής.

E2: Πρόβλεψη Σθένους με Νευρωνικά Δίκτυα

Τώρα θα προσπαθήσετε να κάνετε το καλύτερο μοντέλο χρησιμοποιώντας νευρωνικά δίκτυα.

- Είστε ελεύθεροι να χρησιμοποιήσετε όποια αρχιτεκτονική νευρωνικών δικτύων θέλετε. Προσέξτε την υπερπροσαρμογή.
- Αφού βρείτε το καλύτερο μοντέλο, θα το χρησιμοποιήσετε για να υπολογίσετε το σθένος των κομματιών που δίνονται στο αρχείο `spotify_ids.txt` . Ως μετρική θα πρέπει να χρησιμοποιήσετε το Μέσο Απόλυτο Λάθος (Mean Absolute Error).

Τελικά, ποια ήταν η καλύτερη προσέγγιση, με ή χωρίς νευρωνικά δίκτυα;

Το μοναδικό νευρωνικό δίκτυο της βιβλιοθήκης sklearn που επιλύει πρόβλημα ταξινόμησης είναι το MLPRegressor.

In [139]:
```
Y_test = test_df["valence"].values
```

In [151]:
```
mlpr = make_pipeline(StandardScaler(), MLPRegressor(max_iter = 100))
mlpr.fit(X_train, Y_train)
mlpr_preds = mlpr.predict(X_test)
print(mlpr.score(X_test, Y_test))
```

Out[151]:
```
0.32360038412372893
```

Προκύπτει σχετικά χαμηλή τιμή για την R^2, και ύστερα από πειραματισμό αύξηση των εποχών δεν οδήγησε σε βελτίωση. Θα δοκιμάσω και την προσέγγιση ταξινόμησης, με απλό MLP.

In [154]:
```
Y_train_quant = np.digitize(df["valence"], bins = [x / 10 for x in range(0, 11)])
Y_test_quant = np.digitize(test_df["valence"], bins = [x / 10 for x in range(0, 11)])
```

In [158]:
```
from sklearn.neural_network import MLPClassifier

mlpp = make_pipeline(StandardScaler(), MLPClassifier(max_iter = 1000))
mlpp.fit(X_train, Y_train_quant)

print(mlpp.score(X_test, Y_test_quant))
```

```
0.23235800344234078
```

Πάλι αποτέλεσμα σχεδόν ανούσιο με την προσέγγιση της ταξινόμησης, *στην οποία όμως εστίασε σχεδόν αποκλειστικά το μάθημα όσον αφορά τα νευρωνικά δίκτυα*. Ενδεικτικά, παρακάτω φαίνεται πώς θα προσέγγιζα τα αποτελέσματα ταξινόμησης αν αυτό είχε κάποιο νόημα.

In [160]:
```
from sklearn.metrics import confusion_matrix

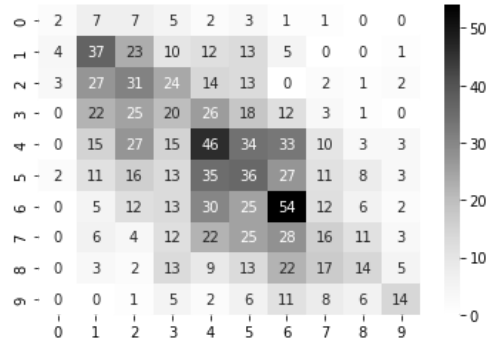
mlpp_preds = mlpp.predict(X_test)
cnfm = confusion_matrix(Y_test_quant, mlpp_preds)
print(classification_report(Y_test_quant, mlpp_preds))
sns.heatmap(cnfm, annot = True, cmap = "Greys", fmt = 'd')
```

```
              precision    recall  f1-score   support

           1       0.18      0.07      0.10        28
           2       0.28      0.35      0.31       105
           3       0.21      0.26      0.23       117
           4       0.15      0.16      0.16       127
           5       0.23      0.25      0.24       186
           6       0.19      0.22      0.21       162
           7       0.28      0.34      0.31       159
           8       0.20      0.13      0.15       127
           9       0.28      0.14      0.19        98
          10       0.42      0.26      0.33        53

    accuracy                           0.23      1162
   macro avg       0.24      0.22      0.22      1162
weighted avg       0.23      0.23      0.23      1162
```

Out[160]: <AxesSubplot:>



Δεύτερη δοκιμή με υψηλότερη κβάντιση

Κάνω μία δεύτερη δοκιμή, αυτήν την φορά χωρίζοντας το διάστημα [0, 1] σε τέσσερα υποδιαστήματα πλάτους 0.25, με τέσσερεις κλάσεις δηλαδή.

In [19]:
```
X_train = df[["danceability", "energy", "key", "loudness", "speechiness", "acousticness", "instrumentalness", "liveness", "tempo"
X_test = test_df[["danceability", "energy", "key", "loudness", "speechiness", "acousticness", "instrumentalness", "liveness", "te
```

In [30]:
```
Y_train_quant = np.digitize(df["valence"], bins = [x / 100 for x in range(0, 125, 25)])
Y_test_quant = np.digitize(test_df["valence"], bins = [x / 100 for x in range(0, 125, 25)])
```

In [32]:
```
mlpp = make_pipeline(StandardScaler(), MLPClassifier(max_iter = 1000))
mlpp.fit(X_train, Y_train_quant)

print(mlpp.score(X_test, Y_test_quant))
```

```
0.4578313253012048
```

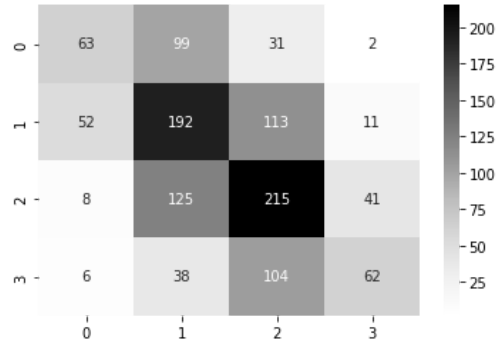
In [33]:
```
    mlpp_preds = mlpp.predict(X_test)
    cnfm = confusion_matrix(Y_test_quant, mlpp_preds)
    print(classification_report(Y_test_quant, mlpp_preds))
    sns.heatmap(cnfm, annot = True, cmap = "Greys", fmt = 'd')
```

```
              precision    recall  f1-score   support

           1       0.49      0.32      0.39       195
           2       0.42      0.52      0.47       368
           3       0.46      0.55      0.50       389
           4       0.53      0.30      0.38       210

    accuracy                           0.46      1162
   macro avg       0.48      0.42      0.44      1162
weighted avg       0.47      0.46      0.45      1162
```

Out[33]: <AxesSubplot:>



Αυξήθηκε η ορθότητα, όμως ακόμη είναι σε πολύ χαμηλό επίπεδο, και τώρα υπάρχει και μεγαλύτερη απώλεια πληροφορίας (οι προβλέψεις έχουν σφάλμα ± 0.125 αντί για ± 0.05).

Τελικά συμπεράσματα

Η μοναδική μέθοδος που παρήγαγε ικανοποιητικά αποτελέσματα ήταν το **δένδρο αποφάσεων**. Οι προσεγγίσεις με ταξινόμηση ήταν εντελώς άστοχες, και το MLPRegressor επίσης ανεπαρκές. Ίσως να βελτιωνόταν λίγο η επίδοση του MLPCLassifier αν υπήρχε ομοιόμορφη κατανομή των κλάσεων στα δεδομένα εκπαίδευσης, το οποίο μπορεί να επιτευχθεί τεχνητά με υπερδειγματοληψία στο train set. Αν συνέχιζα την εργασία, θα επιχειρούσα **πολλαπλή παλινδρόμηση**, εφ' όσον έχω πολλές ανεξάρτητες μεταβλητές, και θα διερευνούσα επίσης αν είναι όντως ανεξάρτητες.

Οδηγίες Υποβολής

Θα πρέπει να υποβάλετε την εργασία σας με τη μορφή σημειωματαρίου (Jupyter notebook), το οποίο θα περιέχει το σύνολο του κώδικα και την ανάλυσή σας. Αυτό σημαίνει ότι το σημειωματάριο θα πρέπει να διαβάζεται και να γίνεται κατανοητό και ως κείμενο (αλλιώς η υποβολή θα ήταν απλώς ένα αρχείο Python). Ο σκοπός δεν είναι μόνο να δείξετε ότι ξέρετε να γράφετε κώδικα, αλλά να εξηγήσετε και να πείσετε τον αναγνώστη. Θα πρέπει να υποβάλετε και όποια αρχεία δεδομένων χρησιμοποιείτε και να εξασφαλίσετε ότι το σημειωματάριο μπορεί να τρέξει χωρίς καμμία μετατροπή σε έναν άλλο υπολογιστή εκτός του δικού σας, *εξαιρουμένου του αρχείου* `charts.zip` το οποίο δεν χρειάζεται να συμπεριλάβετε στην υποβολή σας. Προσέξτε ιδιαιτέρως τα μονοπάτια ανάγνωσης των αρχείων, αυτά πρέπει να είναι σχετικά (relative paths) και όχι απόλυτα (absolute paths).

Κώδικας Τιμής

Συνειδητοποιείτε ότι αυτή είναι ατομική εργασία και θα πρέπει να εργαστείτε μόνοι σας. Μπορείτε βεβαίως να αναζητήσετε βοήθεια στο διαδίκτυο, αλλά αυτή θα πρέπει να αφορά τη χρήση των εργαλείων και ιδιωμάτων, δεν θα πρέπει να αναζητείτε την απάντηση στα ερωτήματα της εργασίας. Μπορείτε να συζητήσετε με άλλους που παρακολουθούν το μάθημα προκειμένου να λύσετε μεταξύ σας απορίες κατανόησης, αλλά όχι μεθόδους επίλυσης. Αναλογιστείτε ότι σκοπός του μαθήματος είναι να αποκομίσετε κάποιες γνώσεις, και όχι να ξοδέψετε απλώς τον χρόνο και τα χρήματά σας.